

开源之夏 2026

VSAG 多核高并发检索性能优化

项目申请书

面向图索引检索链路的可复现基线、瓶颈定位与系统优化

申请人	房泽龙	学校	吉林大学
学历	硕士研究生	专业	软件工程
项目编号	26c520013	项目导师	LSH
邮箱	fangzl24@mails.jlu.edu.cn	电话	13592597573
GitHub	github.com/159753a52	申请日期	2026 年 8 月 16 日

申请摘要

本申请拟在 VSAG 现有框架中建立严格可复现的多核并发检索基线，结合 CPU 硬件计数器、火焰图、锁与 NUMA 分析定位扩展性瓶颈，并以不低于 15% 的中高并发 QPS 提升为核心目标，完成可审查的代码、测试、实验报告和开源贡献。

1. 项目理解与申请目标

1.1 项目背景

VSAG 是面向向量相似性搜索的高性能 C++ 库。本项目关注图索引（以向量为节点、相似关系为边的索引）在多核 CPU、高并发请求下的扩展效率。随着并发度增加，图遍历会产生离散邻居访问，距离计算持续读取向量数据，候选堆和 `visited-list` 频繁更新；这些路径共同受到缓存命中率、内存带宽、同步开销和线程调度的影响。

当并发线程超过物理核心数或跨越 NUMA 节点（同一服务器内的一组 CPU 核心和本地内存）后，QPS（每秒完成的检索请求数）可能不再线性增长，甚至因缓存争用、`false sharing`、远端内存访问和内部/外部线程过度订阅而下降。因此，本项目需要同时解决两类问题：一是建立足够严格的实验方法，区分真实收益与系统噪声；二是根据 `profiling` 证据选择足以支撑目标的热点路径，从而实现不低于 15% 的 QPS 提升。

1.2 检索链路理解

KnnSearch-> 参数与查询上下文 -> 图遍历/visited-list -> 向量读取与距离计算 -> 候选堆维护 -> 结果整理

我将把一次查询拆分为请求准备、路由层搜索、底层图遍历、距离计算、候选维护、可选重排和结果封装等阶段。每个阶段分别记录实际耗时（`wall-clock time`）、CPU 周期、指令数、缓存未命中和分配行为，再结合并发度从 1 增加到 32 时的 QPS 扩展曲线判断瓶颈是计算受限、内存受限还是同步受限，避免仅凭源码直觉实施大范围重构。

1.3 核心目标与验收边界

维度	验收场景	目标	判定方法
吞吐	并发 8/16/32	QPS 提升 >= 15%	同硬件、同索引、同参数的 <code>paired A/B</code> 中位数
召回	全部正式场景	相对基线变化 <= 1%	使用同一真值集与同一 <code>recall</code> 口径
尾延迟	并发 8/16/32	P99 恶化 <= 10%	同时保留 P50/P99 与原始轮次
正确性	单元/并发回归	无错误、死锁和数据竞争	聚焦测试、结果计数、必要时 TSAN
工程交付	3-5 个公开数据集	可复现并可审查	脚本、配置、报告、文档与最小化 PR

范围边界 本项目聚焦多核 CPU 的并发检索效率；所有收益都必须在召回率和 P99 延迟约束内成立，避免以牺牲结果质量或尾延迟换取吞吐。

2. 申请基础与前期预研

2.1 教育与技术基础

我目前就读于吉林大学软件学院软件工程专业硕士，具备软件工程与系统实现背景；本科毕业于郑州大学计算机与人工智能学院软件工程专业。曾获蓝桥杯第十五届软件赛省赛 C/C++ 程序设计大学 A 组一等奖，具备较扎实的 C/C++ 编程和算法基础。

在研究和工程实践中，我熟练使用 Python/C++ 与 PyTorch 分布式底层机制，具有流水线并行调度、CUDA/cuBLAS/cuDNN 调用拦截和多 stream 并发调优经验。虽然 GPU 调度与 CPU 图检索的硬件瓶颈不同，但二者在工作负载拆分、事件同步、并发资源竞争、基线控制和吞吐评测方面具有可迁移的方法论。

2.2 相关研究与工程经验

- 流水线并行训练框架：复现并工程化 GPipe、1F1B、Hanayo、ZeroBubble 等调度策略，实现基于 LD_PRELOAD 的 GPU 算子拦截层；在 4xA100、同模型和 batch 配置下，训练吞吐提升约 12%-18%。
- 大模型检查点压缩研究：提出基于二阶泰勒展开与 HVP 的损伤度量，设计分布感知全局分配框架，在 GPT-2、Pythia 和 ViT 上完成 10%-40% 稀疏度验证。
- 工程能力：能够阅读和修改规模较大的开源工程，使用 Git、CMake、单元测试和基准工具完成实现、回归与提交。

2.3 VSAG 前期预研

为验证项目可行性，我已在申请阶段开展一轮预研。当前工作不作为正式验收结果，而是用于确认调用链、测试工具和优化方向可以落地。预研代码已保存在个人 fork 的候选分支，方便导师审阅。

预研分支： github.com/159753a52/vsag/tree/xlc/hgraph-qps-15

预研内容	已完成事项	当前结论
调用链审计	梳理 HGraph KNN、Basic/ParallelSearcher、visited-list、距离计算和结果封装	已形成热点候选清单
A/B 工具	独立 evaluator/libvsag、共享索引、预热、交替 paired 运行、hash 与动态库校验	16 项 Python 回归通过
低风险优化	参数解析复用、不可变锁快路、候选堆栈化、StandardHeap 预留边界	聚焦单元测试通过
方向性短测	固定随机种子生成 20k 条 32 维 FP32 向量并构建 HGraph；3 对短 A/B，并发 8/16/32	仅用于验证评测链路 with 候选趋势

方向性短测：QPS 提升幅度

20k 合成索引, 3 对短 A/B; 单点超过 15% 不代表正式达标

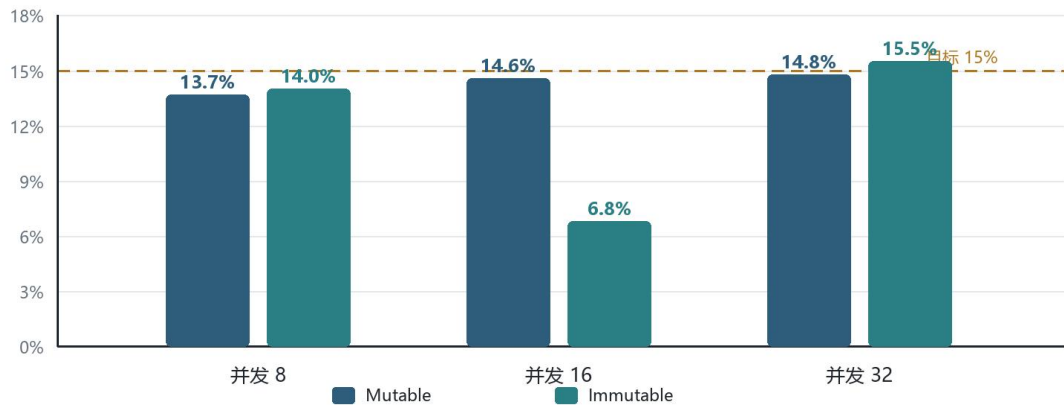


图 1 前期方向性短测（不构成 15% 正式验收结论）

预研判断 单个并发点超过 15% 不能证明正式达标，Immutable 在 16 并发下也存在明显波动。正式实验将从未包含候选优化的对照提交重新编译 baseline，在 3-5 个数据集上保留全部原始 JSON，并完成至少 5 轮 paired A/B。

2.4 预研局限与正式复验计划

当前方向性数据来自固定随机向量构建的 20k 小规模 HGraph 索引和 6 核 12 线程 WSL 环境，16/32 并发存在过度订阅，因此只能说明评测链路和优化候选可以继续验证，不能用于项目验收。

正式主环境已准备为 ecs-7eb4：64 个逻辑 CPU、32 个物理核心、1 个 socket、1 个 NUMA 节点和约 128 GiB 内存。该环境可完成任务要求的 1-32 并发测试与通用 perf profiling；由于没有跨 NUMA 路径且未暴露 uncore/IMC 事件，本项目不会用它宣称跨 NUMA 或内存控制器带宽优化收益。多 NUMA 仅作为额外硬件可用时的扩展验证，不是正式验收前提。

- 重新下载并校验公开数据集，将数据、构建产物和索引放入持久目录，避免临时目录造成实验丢失。
- 分别编译无优化 baseline 与候选分支，确认两个评测程序各自加载对应的 libvsag，并记录文件路径与 SHA-256 指纹。
- 项目详情明确要求覆盖 3-5 个公开或项目已有数据集；优先使用 VSAG benchmark 已覆盖的 SIFT-1M、NYTimes-256、GloVe-100，并补足最终实验矩阵。

3. 技术方案与实施方法

3.1 冻结可复现基线

1. 冻结 `ecs-7eb4` 为正式主环境，记录 32 个物理核心、64 个逻辑 CPU、单 NUMA、约 128 GiB 内存，以及 CPU 型号、编译器、SIMD 指令集和操作系统内核。
2. 使用开启编译优化的 `Release` 构建，为 `baseline/candidate` 分别生成 `evaluator` 与 `libvsag`；通过 `realpath`、`ldd` 和 `SHA-256` 确认两者没有加载同一动态库。
3. 同一数据集只构建一次共享序列化索引；固定 `create/search` 参数、`top-k`、查询集和 `recall` 真值。
4. 固定 CPU affinity、性能 governor 和 OpenMP 线程环境；主实验优先绑定 0,2,4,...,62，每个物理核心只取一个逻辑 CPU，预热后再让 `baseline/candidate` 成对交替运行。

3.2 基准测试设计

正式实验覆盖 1、2、4、8、16、32 个同时执行的独立查询（外层并发），重点考察 8、16、32。每次 `evaluator` 进程先执行 10,000 次不计量预热，再执行 100,000 次计量查询；每个并发至少 5 轮 paired A/B，保留单轮原始 JSON，以中位数作为主结论，并报告执行顺序、有效样本数和失败原因。

- 吞吐指标：QPS、相对提升、相对单线程的扩展效率。
- 延迟指标：平均延迟、P50、P99；不以平均值掩盖尾延迟回归。
- 质量指标：Recall、成功查询数、错误数；必要时增加结果 ID/距离摘要以发现聚合指标盲区。
- 资源指标：CPU 利用率、RSS、分配次数、上下文切换和 CPU 迁移；单 NUMA 主环境记录拓扑与内存页分布，只有额外多 NUMA 环境可用时才比较本地/远端访问。

3.3 Profiling 与瓶颈归因

Profiling 将按“总体现象 -> 热点函数 -> 微架构事件 -> 线程/NUMA 归因”的顺序进行。所有工具数据都与同一轮 QPS、延迟和 `recall` 结果关联，避免在不同负载之间拼接结论。

- `perf stat`：cycles、instructions、IPC、branches、branch-misses、cache-references、cache-misses、context-switches、cpu-migrations。
- `perf record/report` 与火焰图：定位图遍历、距离计算、候选堆、分配器和结果封装的 CPU 占比。
- `perf lock`、锁统计或等效采样：验证 `shared_mutex`、对象池分片和线程池同步是否形成争用。
- `numactl/numastat` 用于确认拓扑和内存页分布；当前云服务器以通用 `perf` 事件为主，只有额外多 NUMA 环境及 PMU 事件可用时才用 `uncore/PCM/LIKWID` 评估远端访存和内存带宽。

3.4 第一性原理判定

收益门槛 QPS 提升 15% 等价于总有效耗时下降约 13.04% ($1 - 1/1.15$)。若候选阶段占比不足 13%，即使理论上把该阶段耗时降为零也无法独立达标，因此每项优化必须先用 `profiling` 或单变量 A/B 证明其耗时占比。

4. 优化路线

4.1 低风险固定开销

首先优化每查询固定发生、且不改变搜索顺序的开销，包括重复 JSON 解析、非逃逸临时对象、候选堆初始容量、结果整理与统计序列化。对于工作区复用，将以 `allocator`、索引实例、图最大度和模式作为 `key`，设置容量上限并保留回退路径，避免跨索引污染与 TLS 无界增长。

4.2 只读路径与锁

当索引构建完成且后续不再写入时，调用 `SetImmutable()` 将索引发布为只读状态，再评估是否可跳过只为防止并发写入而存在的节点锁。`Mutable` 与 `Immutable` 必须分别测量，不能把只读开关本身算作候选版本独有收益；测试将覆盖公开 `SetImmutable/KnnSearch` 入口、并发搜索、重复切换以及序列化后恢复，并配合 TSAN 检查数据竞争。

4.3 缓存、布局与 false sharing

只有在硬件计数器证明缓存失效或 `cache line` 争用显著时，才调整热点计数器、队列游标或 `per-thread` 状态的内存对齐与 `padding`；对于访问密集的邻接表、向量编码和 `visited-list`，将评估连续分块、预取距离与批处理大小。每项布局变化都要同时检查内存占用，防止以大幅空间膨胀换取局部吞吐。

4.4 并发调度与可选 NUMA 扩展

调度候选将由正式 A/B 决定：高外层并发时把查询内部线程数固定为 1；低外层并发且单次查询较重时再尝试 2 个内部线程。当前单 NUMA 主环境足以完成多核验收；若后续获得多 NUMA 机器，再附加比较本地分配、节点内绑定和查询路由，并将结果作为适用边界，而不是必做验收项。

4.5 方案选择与回退

每轮只引入一个主要变量，先运行聚焦测试，再执行短 A/B；无稳定收益或违反召回/P99 约束的改动立即回退。达到门槛的方案才进入完整数据集矩阵，并以独立 `commit` 保持代码可审查。

4.6 优化决策与变更管理

每个候选都记录“证据、假设、实现、测试、A/B、结论”，只有热点占比足以支撑目标时才进入实现；每轮验证后单独 `commit` 并推送个人 `fork`，保留可回退历史。

- 进入下一阶段（Go）：测试通过、QPS 收益稳定，`recall/P99` 满足门槛。
- 调整后重测（Revise）：方向正确但收益不足，根据 `perf` 变化调整后复验。
- 停止并回退（Stop）：收益不可重复、范围过窄或存在正确性风险。

5. 正确性、工程质量与验收

5.1 正确性与线程安全

- 保持公开 API/ABI、搜索参数语义、序列化格式和结果排序兼容。

- 为缓存边界、异常输入、线程隔离、mutable/immutable 切换和超大 top-k 增加回归测试。
- 检查对象生命周期、allocator 归属和跨线程引用；必要时使用 TSAN 或受控并发测试。
- 在性能报告中同时保留 sample、successful query、error 和 recall，失败轮次不进入收益中位数。

5.2 数据集与实验矩阵

项目详情明确要求覆盖 3-5 个公开或项目已有数据集。实验优先使用 VSAG benchmark 已覆盖的 SIFT-1M、GloVe-100、NYTimes-256，并结合数据可用性补足最终矩阵。每个数据集固定索引参数和搜索参数，记录数据集哈希与共享索引哈希；同一组参数下分别完成 Mutable/Immutable 及 1-32 并发测量。

实验层级	目的	运行规模	进入下一阶段条件
聚焦测试	验证行为与边界	相关单元/并发用例	全部通过，格式与静态检查通过
短 A/B	筛选方向	小数据、3-5 个 pair	QPS 有稳定正收益且质量指标无回归
正式单数据集	判断核心目标	100k 查询、>=5 个 pair	8/16/32 全部满足验收阈值
跨数据集	验证泛化性	3-5 个公开数据集	说明收益范围、例外和资源代价

5.3 交付物

- 可重复构建的 baseline/candidate 分支、测试和最小化 PR。
- 并发 A/B harness、固定配置、运行命令、原始 JSON 与汇总报告。
- 优化前后 QPS/扩展效率曲线、P50/P99、recall、硬件计数器和火焰图。
- 技术报告：根因、方案设计、收益、适用条件、局限性与后续工作。

6. 三个月开发计划

阶段	时间	主要工作	阶段交付
基线冻结	第 1-2 周	服务器环境、依赖与双版本构建；数据集/索引准备；1-32 并发基线	环境清单、基线报告、可复现命令
瓶颈定位	第 3-4 周	perf stat/record、火焰图、锁/分配/带宽/NUMA 采样；建立耗时模型	热点排序、证据链与优化候选评审
优化一	第 5-6 周	固定开销、查询工作区、只读锁路径；单元与短 A/B	第一个可审查优化提交
优化二	第 7-8 周	调度、批处理或缓存布局；根据 profiling 决定是否处理 false sharing	第二轮候选与阶段性能报告
泛化与边界	第 9-10 周	扩展至 3-5 个数据集；比较数据分布、Mutable/Immutable 和参数边界；多 NUMA 仅在硬件可用时附加测试	跨数据集收益与适用边界报告
收尾交付	第 11-12 周	完整回归、代码整理、文档、图表、PR 与最终报告	最终 PR、复现实验包和项目总结

7. 风险识别与应对

风险	可能影响	应对措施
----	------	------

风险	可能影响	应对措施
基准噪声	误判 15% 收益	固定频率/亲和性，预热，paired 交替，多轮中位数并保存原始数据
内存不足	ODescent 构建失败	使用约 128 GiB 正式服务器并预估构建峰值；使用持久目录、分离构建与 benchmark，记录峰值 RSS
优化只适配单一场景	跨数据集收益下降	覆盖 3-5 个数据集，明确 Mutable/Immutable、top-k/ef 与拓扑边界
无锁/复用引入竞态	结果错误或崩溃	生命周期审计、RAII、容量回退、并发回归与 TSAN
P99 或 recall 回归	无法通过验收	将质量指标设为硬门槛，逐轮单变量 A/B，失败即回退

8. 预期成果与个人期望

我希望通过项目进一步学习多核性能分析和向量检索工程，并形成可以由导师和社区复核的实验记录：基线如何建立、瓶颈如何定位、优化为什么有效、在哪些硬件和参数下成立、代价是什么。我也将系统深化 CPU 缓存层级、硬件计数器、内存带宽、false sharing 和 NUMA 等知识，并把已有的并发调度与吞吐优化经验迁移到成熟的开源 C++ 向量检索工程中。

项目结束后，我计划继续参与 VSAG 的性能测试与检索链路维护，推动评测脚本/框架（harness）、文档和可复现实验配置长期留在社区中，使后续优化能够建立在统一、可信的测量口径上。

参考链接

- 开源之夏项目页面: <https://summer.ospp.ac.cn/org/prodetail/26c520013?lang=zh&list=pro>
- VSAG 官方仓库: <https://github.com/antgroup/vsag>
- VSAG 官方文档: <https://vsag.io/docs>
- 前期预研候选分支: <https://github.com/159753a52/vsag/tree/xlc/hgraph-qps-15>
- 前期短测汇总记录: <https://github.com/159753a52/vsag/blob/xlc/hgraph-qps-15/docs/2026-08-15-多核高并发检索优化执行记录.md>